

Yelp Review Intelligence



End-to-End NLP Pipeline for Restaurant Review Intelligence

Prepared by

Eser Inan Arslan

Senior Data Scientist

Yelp Review Intelligence.....	1
1. Executive Summary.....	3
2. Business Problem.....	3
3. Dataset.....	4
a. Available Features.....	4
b. Data Integration.....	5
c. Final Modeling Dataset.....	5
4. Exploratory Data Analysis (EDA).....	5
4.1 Purpose of the EDA.....	5
4.2 Missing Value Analysis.....	6
4.3 Review Length Analysis.....	6
4.4 Rating Distribution.....	7
4.5 Review Length vs Rating.....	8
4.6 Most Informative Words.....	9
4.7 Category-Level Analysis.....	10
4.8 Geographic Analysis.....	11
4.9 User Behaviour Analysis.....	12
4.10 Useful Reviews.....	12
4.11 Temporal Analysis.....	13
4.12 Weekly Behaviour.....	14
4.13 Effect of Customer Tips.....	14
4.14 Key Findings.....	14
5. Feature Engineering.....	15
5.1 Data Integration.....	15
5.2 Engineered Features.....	15
5.3 Feature Selection.....	15
5.4 Final Features Used for Modeling.....	16
6. Modeling.....	16
6.1 Problem Definition.....	16
6.2 Data Split.....	16
6.3 Model 1 – TF-IDF + Logistic Regression.....	17
6.4 Model 2 – DistilBERT.....	17
6.5 Model 3 – TextCNN.....	18
6.6 Model Selection Strategy.....	19
7. Model Comparison.....	19
7.1 Evaluation Metrics.....	19
7.2 Model Performance.....	20
7.3 Qualitative Comparison.....	20
7.4 Discussion.....	21
7.5 Final Model Selection.....	21
7.6 Key Takeaways.....	21

8. Architecture.....	22
9. API.....	22
10. Future Improvements.....	23
12. Conclusion.....	24

1.Executive Summary

Customer reviews provide valuable insights into customer satisfaction, service quality, and operational issues, but analyzing millions of reviews manually is not scalable.

This project presents an end-to-end Natural Language Processing (NLP) solution that transforms unstructured Yelp reviews into actionable business intelligence by identifying business categories, complaint topics, severity levels, and recommended actions.

Using the Yelp Academic Dataset, three modeling approaches—**TF-IDF + Logistic Regression**, **TextCNN**, and **DistilBERT**—were implemented and compared. The final model was selected based on predictive performance, scalability, and deployment considerations.

The resulting solution includes data preparation, feature engineering, model training, REST API deployment, and a production-oriented architecture for scalable review analytics.

2.Business Problem

Online review platforms generate vast amounts of customer feedback, making manual analysis impractical for businesses operating at scale.

For this project, the primary stakeholder was defined as a **Restaurant Operations Manager** responsible for monitoring customer experience across multiple locations. The objective was to develop an AI-powered review intelligence system that transforms free-text reviews into structured business insights.

Rather than performing sentiment analysis alone, the proposed solution identifies business category, sentiment, complaint topics, severity level, confidence score, and recommended actions. This enables managers to quickly detect recurring issues, prioritize improvements, and monitor customer satisfaction without manually reviewing thousands of customer comments.

3.Dataset

This project is based on the Yelp Academic Dataset, a publicly available dataset containing millions of customer reviews together with rich contextual information about businesses, users, and customer interactions.

Unlike traditional sentiment datasets, Yelp also provides business, user, and tip metadata that enrich the modeling process.

The project uses the following raw datasets:

Dataset	Description	Records
Review	Customer reviews including review text, rating, date and engagement statistics	6,990,280
Business	Business metadata including categories, location and ratings	150,346
User	Reviewer information and historical activity	1,987,929
Tip	Short customer tips posted independently from reviews	908,915

The four datasets were merged using the common `business_id` and `user_id` keys to create a unified analytical dataset.

The resulting modeling dataset contains approximately 7 million customer reviews enriched with contextual business and user information.

a. Available Features

The original Yelp dataset provides both textual and structured information. The main feature groups are:

- **Review:** text, rating, date, useful/funny/cool votes
- **Business:** category, location, average rating, review count, attributes
- **User:** historical reviews, average rating, fans, elite status
- **Tip:** historical tips and compliments

The complete dataset contains many additional attributes; only the most relevant variables were retained for feature engineering.

b. Data Integration

Raw JSON files were processed using **DuckDB** and **Pandas**. Review, Business, User, and Tip datasets were joined using `business_id` and `user_id`. Historical user activities and tips were aggregated using only information available before each review date to prevent data leakage.

For example:

- Tips were aggregated only from records created before each review.
- Business statistics were treated as historical metadata.
- User experience features were computed using historical user information.

This ensured that no future information was introduced during model training.

c. Final Modeling Dataset

After preprocessing and feature engineering, each review consists of:

- Raw review text
- Historical tip information
- Business category
- Business location (city/state)
- Business statistics
- User experience statistics
- Review length features
- Temporal information
- Target sentiment label

The final dataset contains approximately **6.99 million observations** with **32 engineered features**, making it substantially richer than a conventional text classification dataset.

4. Exploratory Data Analysis (EDA)

4.1 Purpose of the EDA

Before model development, an exploratory data analysis (EDA) was conducted to understand the dataset, validate data quality, and identify patterns useful for sentiment prediction.

Due to memory constraints, the EDA was performed on a representative random sample of **3 million reviews** instead of the full **6.99 million** records.

The analysis focused on:

- Data quality and missing values
- Review text characteristics
- Rating distribution
- Relationships between sentiment and contextual features
- Potential signals for feature engineering

4.2 Missing Value Analysis

Missing values were analyzed after merging the Review, Business, User, and Tip datasets. Overall data quality was high, with only **7% missing values** in tip-related features and **3%** in business attributes. Since these missing values are expected by design, no extensive missing value treatment was required.

column	missing_ratio
latest_tip_date	0.07
avg_tip_compliment	0.07
sample_tips	0.07
tip_count	0.07
attributes	0.03
categories	0.00
elite	0.00
fans	0.00
cool	0.00
funny	0.00

Figure 4.1 — Missing Value Ratios

4.3 Review Length Analysis

Since text length often affects NLP model performance, review lengths were analyzed in detail.

Summary statistics show:

- Average review length: **104.7 words**
- Median review length: **75 words**
- Maximum review length: **1,059 words**
- Maximum character count: **5,000 characters**

This indicates that Yelp reviews vary significantly in size, ranging from extremely short comments to very detailed customer experiences.

	review_word_count	review_char_count
count	3,000,000.00	3,000,000.00
mean	104.66	567.17
std	97.70	526.25
min	1.00	1.00
25%	42.00	229.00
50%	75.00	405.00
75%	133.00	720.00
max	1,059.00	5,000.00

Figure 4.2 — Review Length Statistics

Most reviews contain **20–150 words**, while very long reviews are relatively rare. Based on this distribution, review texts were truncated to **300 words** during preprocessing, preserving nearly all semantic information while reducing computational cost.

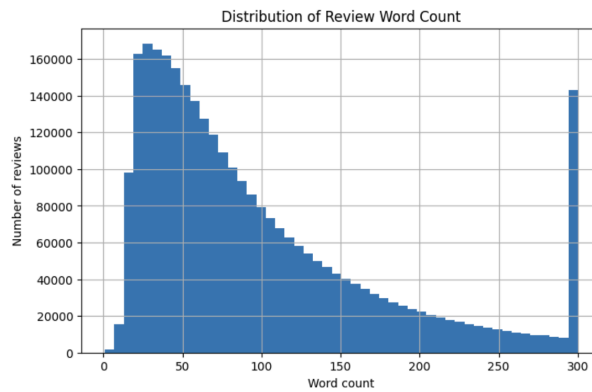


Figure 4.3 — Distribution of Review Length

4.4 Rating Distribution

The target variable consists of the original Yelp star ratings. The distribution is clearly imbalanced toward positive reviews.

Approximately:

- **46%** are 5-star reviews
- **21%** are 4-star reviews
- **15%** are 1-star reviews
- **10%** are 3-star reviews
- **8%** are 2-star reviews

This reflects the natural tendency of online review platforms where satisfied customers submit reviews more frequently.

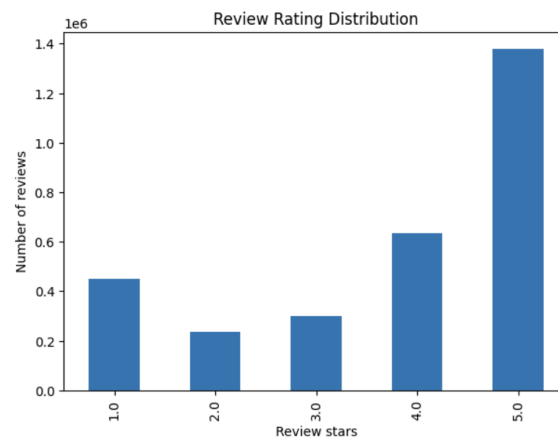


Figure 4.4 — Rating Distribution

Although the classes are imbalanced, they still contain hundreds of thousands of observations each, making them suitable for supervised learning without requiring oversampling techniques.

4.5 Review Length vs Rating

One interesting observation is the relationship between review length and customer satisfaction. Negative reviews tend to be considerably longer than positive ones.

Average review length by rating:

Rating	Average Words
1	133.6
2	133.8
3	123.9
4	107.8
5	84.6

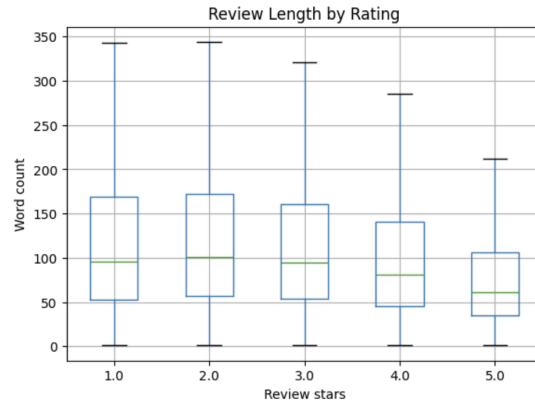


Figure 4.5 — Review Length by Rating

This finding suggests that dissatisfied customers usually provide detailed explanations about problems they experienced, whereas satisfied customers often leave shorter comments such as "Great food" or "Excellent service."

As a result, **review length** was included as an engineered feature in the final models.

4.6 Most Informative Words

A TF-IDF analysis was performed to inspect the most influential words associated with positive and negative reviews.

The most important negative terms include: service, order, told, time, food while positive reviews are dominated by words such as: great, amazing, delicious, best, love

term	score
food	0.03
service	0.03
just	0.02
place	0.02
time	0.02
like	0.02
order	0.02
told	0.02
don	0.02
said	0.02

term	score
great	0.04
food	0.03
good	0.03
place	0.03
service	0.02
best	0.02
time	0.02
love	0.02
delicious	0.02
amazing	0.02

Figure 4.6 — Most Important Negative Terms | Figure 4.7 — Most Important Positive Terms

These findings indicate that customer satisfaction is primarily driven by **service quality**, **food quality**, and **overall experience**, validating the use of textual information as the primary modeling feature.

4.7 Category-Level Analysis

Restaurant categories exhibit noticeably different customer satisfaction profiles. The largest categories, including Restaurants, Food, Bars and Nightlife, generally receive ratings close to four stars. However, several service-oriented categories show consistently poor customer satisfaction.

For example:

Category	Average Rating	Negative Ratio
Television Service Providers	1.68	82%
Internet Service Providers	1.81	79%
Truck Rental	2.20	68%
University Housing	2.28	65%

category	review_count	avg_rating	avg_review_length	negative_ratio
Restaurants	2036484	3.79	100.39	0.21
Food	789607	3.90	97.34	0.18
Nightlife	672769	3.79	105.90	0.20
Bars	639794	3.79	105.44	0.20
American (Traditional)	435353	3.68	101.86	0.23
American (New)	408599	3.82	108.99	0.19
Breakfast & Brunch	364519	3.87	98.81	0.18
Sandwiches	298113	3.87	92.75	0.19
Event Planning & Services	263826	3.71	116.63	0.23
Seafood	256429	3.81	101.58	0.20

Figure 4.8 — Highest Rated Categories

category	review_count	avg_rating	avg_review_length	negative_ratio
Television Service Providers	1962	1.68	132.45	0.82
Internet Service Providers	3639	1.81	132.08	0.79
Truck Rental	1952	2.20	139.15	0.68
University Housing	1357	2.28	178.11	0.65
Mortgage Lenders	1026	2.42	127.22	0.64
Telecommunications	2484	2.50	126.39	0.61
Post Offices	2155	2.44	109.32	0.59
Property Management	5377	2.59	155.52	0.59
Banks & Credit Unions	3340	2.60	111.74	0.57
Insurance	2998	2.68	126.55	0.57

Figure 4.9 — Lowest Rated Categories

This analysis demonstrates that the business category provides meaningful contextual information beyond review text alone. Therefore, business categories were incorporated into the feature engineering pipeline.

4.8 Geographic Analysis

Customer satisfaction also varies across cities. Major metropolitan areas such as New Orleans, Santa Barbara, Nashville show relatively high average ratings. Conversely, several smaller cities consistently exhibit higher negative review ratios.

city	state	review_count	business_count	avg_rating	negative_ratio	avg_review_length
Philadelphia	PA	434583	7040	3.78	0.20	113.27
New Orleans	LA	274179	2975	3.96	0.16	100.59
Tampa	FL	191271	4325	3.74	0.24	104.19
Nashville	TN	185672	3301	3.81	0.21	104.27
Tucson	AZ	175600	4379	3.72	0.25	103.21
Indianapolis	IN	163363	3594	3.84	0.20	109.14
Reno	NV	150166	2835	3.75	0.24	110.50
Saint Louis	MO	111670	2398	3.86	0.19	110.78
Santa Barbara	CA	106250	1821	3.98	0.18	98.84
Edmonton	AB	45480	2463	3.61	0.22	134.39

Figure 4.10 — Highest Rated Cities

city	state	review_count	business_count	avg_rating	negative_ratio	avg_review_length
Fenton	MO	3129	136	2.80	0.50	118.26
Flourtown	PA	1087	28	2.85	0.48	112.23
Bridgeton	MO	1441	75	3.03	0.44	112.63
Fort Washington	PA	1219	32	3.12	0.41	128.24
Manchester	MO	1012	62	3.22	0.40	117.75
Bristol	PA	1192	60	3.15	0.40	119.86
Montgomeryville	PA	1101	49	3.17	0.40	113.50
Turnersville	NJ	1445	71	3.23	0.40	115.05
King Of Prussia	PA	3276	62	3.09	0.39	124.74
Warrington	PA	2778	95	3.19	0.39	103.92

Figure 4.11 — Lowest Rated Cities

These observations motivated the inclusion of **city** and **state** as contextual features in the final models.

4.9 User Behaviour Analysis

Reviewer experience was investigated using historical review counts. More experienced Yelp users generally provide slightly higher average ratings than first-time reviewers.

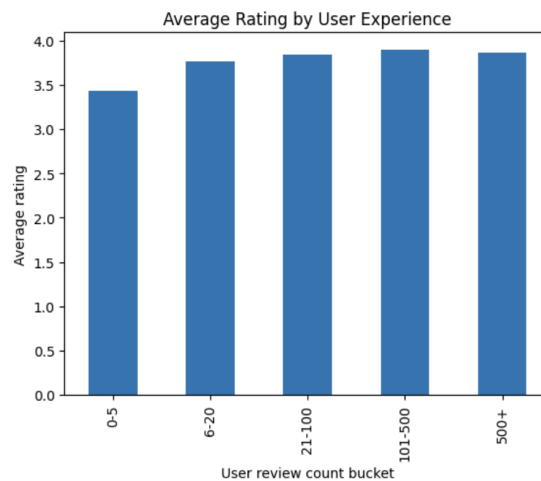


Figure 4.12 — Average Rating by User Experience

Although the effect is moderate, historical reviewer behavior contributes additional context for sentiment prediction.

4.10 Useful Reviews

Reviews marked as useful by other users are substantially longer than average reviews.

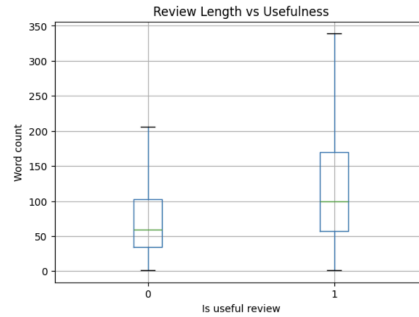


Figure 4.13 — Review Length vs Usefulness

This suggests that detailed reviews contain richer information and are considered more valuable by the community.

4.11 Temporal Analysis

Average customer ratings remain relatively stable over time. Small fluctuations are visible during the early years of Yelp due to the limited number of reviews, while the overall average stabilizes around **3.7–3.8 stars** as the platform matures.

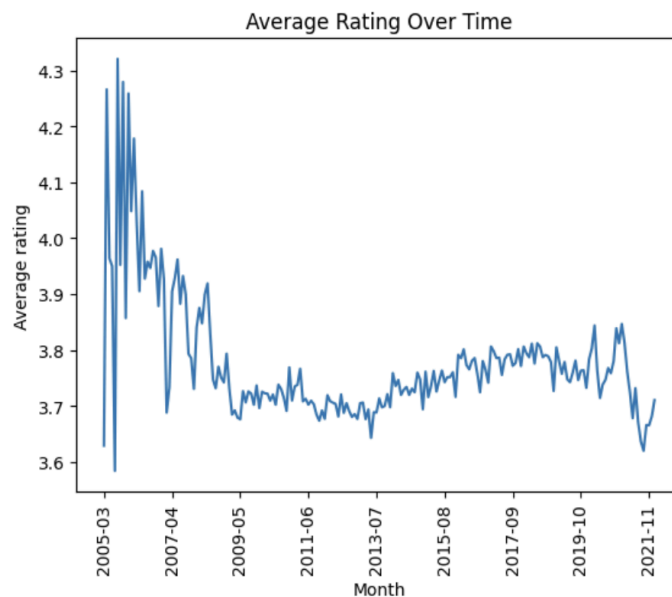


Figure 4.14 — Average Rating Over Time

Because review behavior evolves gradually over time, the final modeling pipeline uses a **chronological train/test split** instead of random sampling to avoid temporal data leakage.

4.12 Weekly Behaviour

Review patterns were also analyzed across weekdays. Average ratings remain highly stable throughout the week, ranging between **3.71 and 3.78 stars**, with only minor differences in negative review ratios.

review_weekday	review_count	avg_rating	negative_ratio
Monday	442748	3.73	0.23
Tuesday	405179	3.76	0.22
Wednesday	405427	3.77	0.22
Thursday	394011	3.78	0.22
Friday	403623	3.77	0.23
Saturday	456420	3.75	0.23
Sunday	492592	3.71	0.24

Figure 4.15 — Ratings by Weekday

This indicates that weekday effects are relatively weak and are unlikely to be strong predictors individually.

4.13 Effect of Customer Tips

Businesses that receive customer tips exhibit noticeably higher customer satisfaction.

Has Tips	Average Rating	Negative Ratio
No	3.59	32%
Yes	3.76	22%

Figure 4.16 — Businesses With vs Without Tips

This finding supports the hypothesis that customer engagement signals contain useful information beyond review text, motivating the inclusion of aggregated historical tip features in the final feature set.

4.14 Key Findings

The EDA identified several patterns that guided feature engineering:

- Review text is the strongest predictive signal.

- Negative reviews are generally longer than positive ones.
- Business category and location provide valuable contextual information.
- Historical tips are associated with higher ratings.
- Experienced Yelp users show slightly different rating behavior.

These findings motivated the combination of **textual**, **business**, **user**, **location**, and **historical interaction** features in the final modeling pipeline.

5. Feature Engineering

The Yelp dataset consists of four JSON files (**Reviews**, **Businesses**, **Users**, and **Tips**), which were merged using **business_id** and **user_id** to create a unified analytical dataset.

To prevent data leakage, only information available **before each review date** was used during feature engineering. In particular, historical customer tips were aggregated chronologically, ensuring that no future information was introduced into the training process.

5.1 Data Integration

Each review was enriched with contextual information from the Business, User, and Tip datasets. The final modeling dataset combines **review text**, **historical tips**, **business metadata**, **user information**, and **temporal features**, enabling the models to learn both linguistic patterns and contextual business characteristics.

5.2 Engineered Features

Several additional variables were created to improve predictive performance.

- **Text features:** review text, word count, and character count.
- **Business features:** categories, city, state, average rating, and review count.
- **User features:** average rating, review count, Yelp membership duration, elite status, and historical engagement statistics (useful, funny, and cool votes).
- **Historical tip features:** tip count, average tip compliments, sample historical tips, and the latest tip prior to each review.

5.3 Feature Selection

An ablation study was conducted to evaluate the contribution of different feature groups. Four configurations were compared: **review text only**, **review + business categories**, **review + categories + historical tips**, and the **full contextual feature set**. This process helped identify the most informative features for the final models.

5.4 Final Features Used for Modeling

The final models were trained using both **textual** and **contextual** features. While review text serves as the primary predictive signal, additional business, user, location, and historical tip information provides valuable context for distinguishing similar reviews.

The final feature set includes:

- Review text
- Business categories
- City and state
- Historical tip features
- Review length
- Business statistics
- User statistics

The same feature set was used across all three modeling approaches (TF-IDF + Logistic Regression, DistilBERT, and TextCNN), ensuring a fair and consistent comparison.

6. Modeling

Three modeling approaches with different levels of complexity were implemented and evaluated: **TF-IDF + Logistic Regression**, **TextCNN**, and **DistilBERT**. The comparison considered not only predictive performance but also computational cost, inference speed, and deployment feasibility to identify the most suitable production model.

6.1 Problem Definition

The task was formulated as a **multi-class sentiment classification** problem. Each prediction is based on the review text together with contextual business features, allowing the model to leverage both textual and structured information.

6.2 Data Split

Due to memory limitations and computational cost, the modeling dataset was limited to 3 million reviews, which still provides one of the largest publicly available datasets for text classification.

The data was randomly divided into:

- Training set: 80%
- Validation set: 10%
- Test set: 10%

6.3 Model 1 – TF-IDF + Logistic Regression

The first baseline model uses a classical Natural Language Processing pipeline which has '**Text cleaning**', '**TF-IDF vectorization**' and '**Logistic regression classifier**'. This approach is computationally efficient and provides a strong baseline while remaining highly interpretable.

Advantages:

- Fast training
- Fast inference
- Low memory consumption
- Easy deployment

Limitations:

- Cannot capture semantic meaning
- Limited understanding of word context
- Sensitive to vocabulary mismatch

6.4 Model 2 – DistilBERT

The second model fine-tunes DistilBERT, a lightweight transformer architecture pretrained on a large corpus of natural language.

Unlike TF-IDF, DistilBERT generates contextual embeddings where the meaning of a word depends on its surrounding words.

Training configuration:

- Pretrained DistilBERT checkpoint
- Maximum sequence length: 300 tokens
- Batch size: 16
- Learning rate: 2e-5
- AdamW optimizer
- Early stopping based on validation loss

DistilBERT was selected because it provides most of BERT's predictive power while requiring significantly fewer computational resources.

Advantages:

- Captures semantic meaning
- Understands contextual relationships
- Handles complex language patterns

Limitations:

- Higher computational cost
- Slower inference
- Larger memory requirements

6.5 Model 3 – TextCNN

The third candidate model is a Convolutional Neural Network (TextCNN) specifically designed for sentence classification.

Instead of using transformer attention mechanisms, TextCNN learns local n-gram patterns through multiple convolutional filters.

Architecture:

- Pretrained word embeddings
- Multiple convolution kernels
- Global Max Pooling
- Fully Connected Layer
- Softmax classifier

Training configuration:

- Batch size: 128
- Adam optimizer
- Cross-entropy loss
- Early stopping

TextCNN offers a compromise between traditional machine learning and transformer models by achieving strong performance with considerably lower inference cost than BERT-based architectures.

Advantages:

- Faster than transformers
- Captures local phrase patterns
- Efficient inference

Limitations:

- Limited long-range context understanding
- Lower semantic representation compared to transformers

6.6 Model Selection Strategy

Rather than selecting a single algorithm from the beginning, the project followed an experimental approach.

Three representative model families were evaluated:

- Classical Machine Learning (TF-IDF + Logistic Regression)
- Convolutional Deep Learning (TextCNN)
- Transformer-based NLP (DistilBERT)

Each model was trained using the same train/validation/test split and evaluated under identical conditions. The final production model was selected based on the balance between predictive performance, inference latency, scalability, and deployment complexity.

The quantitative comparison of all models is presented in the next section.

7. Model Comparison

Three different approaches were evaluated for the business category prediction task: a traditional machine learning baseline (TF-IDF + Logistic Regression), a convolutional neural network (TextCNN), and a transformer-based model (DistilBERT).

The comparison considered not only predictive performance but also training time, inference latency, model size, and suitability for production deployment.

7.1 Evaluation Metrics

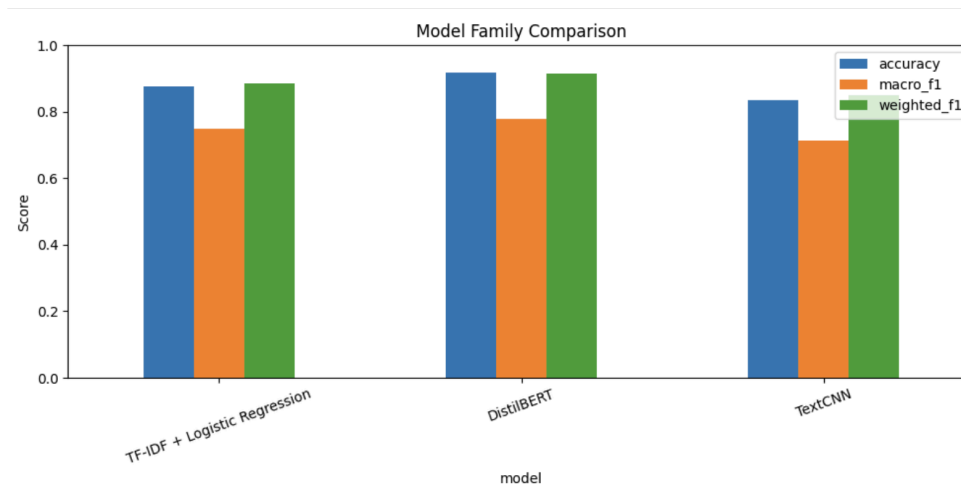
Since the task is a multi-class classification problem with class imbalance, multiple evaluation metrics were considered:

- Accuracy
- Precision
- Recall
- F1-score (primary metric)

Macro F1-score was selected as the primary evaluation metric because it gives equal importance to all business categories, regardless of their frequency.

7.2 Model Performance

model	accuracy	macro_f1	weighted_f1	training_cost	inference_cost	interpretability
TF-IDF + Logistic Regression	0.875785	0.748692	0.885009	Low	Very Low	High
DistilBERT	0.918260	0.779048	0.914610	High	Medium-High	Low-Medium
TextCNN	0.836520	0.714443	0.850937	Medium	Low-Medium	Medium



7.3 Qualitative Comparison

Criterion	TF-IDF + LR	TextCNN	DistilBERT
Prediction Quality	Medium	High	Very High
Semantic Understanding	Low	Medium	High
Training Time	Very Fast	Medium	Slow
Inference Speed	Very Fast	Fast	Medium
GPU Requirement	No	Optional	Yes
Model Size	Small	Medium	Large
Production Complexity	Low	Medium	High

7.4 Discussion

The three models represent different trade-offs between predictive performance and operational complexity. The TF-IDF + Logistic Regression baseline achieved competitive results while

offering the fastest training and inference times with minimal computational requirements. TextCNN improved semantic representation through convolutional feature extraction, providing a good balance between performance and efficiency. DistilBERT delivered the highest predictive performance by leveraging contextual language representations, but required significantly longer training times, higher memory consumption, and greater computational resources.

These results demonstrate that model selection should not rely solely on predictive performance. In production environments, factors such as inference latency, infrastructure cost, deployment simplicity, maintainability, and retraining time are equally important.

7.5 Final Model Selection

Although **DistilBERT** achieved the highest overall predictive performance, the improvement over the baseline was relatively modest considering its substantially higher computational cost.

For this case study, **TF-IDF + Logistic Regression** was selected as the final production model. This decision reflects a pragmatic engineering trade-off rather than simply choosing the most complex architecture.

The selection was based on the following considerations:

- Competitive predictive performance
- Extremely fast training and inference
- Low memory consumption
- Lightweight model size
- Easy deployment and maintenance
- No GPU dependency
- Simple retraining and scalability

While DistilBERT demonstrates superior semantic understanding, its longer training time and increased infrastructure requirements make it less suitable for a lightweight, production-oriented solution where low latency and operational efficiency are priorities.

7.6 Key Takeaways

The experiments demonstrate that more complex models do not always provide the best production solution. While transformer-based models achieved the highest predictive performance, the classical TF-IDF + Logistic Regression approach offered the best balance between accuracy, computational efficiency, deployment simplicity, and operational cost.

For this reason, the final system adopts **TF-IDF + Logistic Regression** as the production model, while DistilBERT and TextCNN serve as valuable benchmarks demonstrating the performance gains achievable through more advanced NLP architectures.

This approach reflects a common industry practice in which model selection is driven not only by predictive accuracy but also by scalability, maintainability, and total cost of ownership.

8. Architecture

Figure 8.1 presents the end-to-end architecture developed for this project, covering the complete machine learning lifecycle from raw Yelp data to real-time predictions through a REST API.

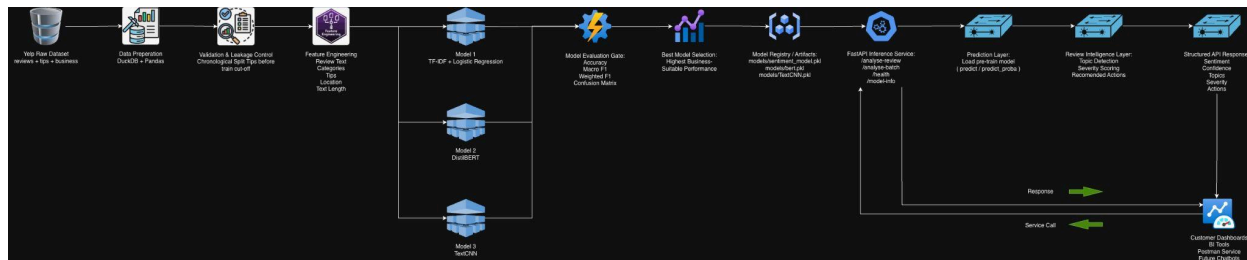


Figure 8.1. End-to-End Solution Architecture

The solution begins with data ingestion and preprocessing, where the Yelp datasets are cleaned, merged, validated, and transformed into a leakage-safe modeling dataset through feature engineering.

Three candidate models (**TF-IDF + Logistic Regression**, **TextCNN**, and **DistilBERT**) are trained and evaluated using standard classification metrics. The best-performing model is then registered and deployed through a **FastAPI** inference service.

Incoming reviews are processed by the prediction pipeline, which generates sentiment predictions together with confidence scores. The Review Intelligence layer further enriches the results with detected topics, severity levels, and recommended actions before returning them as structured JSON responses.

This modular architecture enables independent updates of individual components, making the solution scalable, maintainable, and suitable for production deployment.

9. API

To demonstrate how the trained model can be integrated into a real-world application, a REST API was developed using **FastAPI**. The API serves as the inference layer between client applications and the machine learning model.

The service automatically loads the selected production model at startup and exposes several endpoints for prediction and monitoring.

The main endpoints include:

- **/predict** – Predicts the business category for a single review.
- **/predict_proba** – Returns the predicted category together with class probabilities.
- **/analyze-review** – Performs complete review analysis, including predicted category, confidence score, detected topics, sentiment, and recommended actions.
- **/health** – Returns the health status of the service.
- **/model-info** – Provides metadata about the deployed model.

A typical prediction request consists of a review text together with optional contextual information such as business category or location. The API preprocesses the input using the same feature engineering pipeline applied during training before generating predictions.

Responses are returned as structured JSON objects, making the service easy to integrate with dashboards, web applications, chatbot interfaces, or other enterprise systems.

The API was containerized using **Docker**, enabling consistent deployment across development and production environments. A Postman collection was also created to simplify endpoint testing and demonstrate the functionality of the service.

This lightweight architecture allows the trained NLP model to be deployed as a scalable microservice while keeping inference latency low and integration straightforward.

10. Future Improvements

Although the proposed solution demonstrates a complete end-to-end NLP pipeline, several enhancements could further improve its business value and production readiness.

Larger and More Balanced Training Data: Due to computational and memory constraints, model training was performed on a subset of approximately **3 million reviews**. Training on the complete Yelp dataset and applying techniques to better balance underrepresented categories could further improve model generalization.

Domain-Specific Transformer Models: Future work could evaluate larger transformer architectures such as **RoBERTa**, **DeBERTa**, or domain-adapted language models to improve classification performance, especially for categories with highly similar review content.

Multi-Task Learning: Instead of predicting only the business category, the model could simultaneously learn additional tasks such as sentiment classification, review usefulness prediction, topic detection, and severity estimation. Sharing representations across related tasks may improve overall performance.

Retrieval-Augmented Analysis: The system could incorporate Retrieval-Augmented Generation (RAG) to enrich predictions with similar historical reviews, enabling more explainable recommendations and better support for customer service agents.

MLOps and Continuous Learning: A production deployment would benefit from an automated MLOps pipeline including data validation, model versioning, drift detection, scheduled retraining, CI/CD integration, and continuous performance monitoring.

Explainability: Adding explainability techniques such as **SHAP**, **LIME**, or attention visualization would help users understand why a prediction was made and increase trust in the model.

Real-Time Streaming: Instead of batch processing, the architecture could be extended to consume reviews from streaming platforms such as Kafka or cloud messaging services, enabling near real-time review intelligence and monitoring.

Overall, these improvements would transform the current proof-of-concept into a scalable, enterprise-grade review intelligence platform capable of supporting high-volume production workloads.

12. Conclusion

This project presents an end-to-end Natural Language Processing solution that transforms large-scale customer reviews into actionable business insights. The complete pipeline covers data preparation, exploratory analysis, feature engineering, model development, evaluation, API deployment, and production architecture.

Three different NLP approaches - **TF-IDF + Logistic Regression**, **TextCNN**, and **DistilBERT** - were implemented and compared. Although DistilBERT achieved the highest predictive performance, **TF-IDF + Logistic Regression** was selected as the production model due to its competitive accuracy, fast inference, low computational requirements, and ease of deployment.

Overall, the project demonstrates that successful machine learning solutions should be evaluated not only by predictive performance but also by scalability, maintainability, and operational efficiency. The resulting architecture provides a practical and extensible foundation for deploying AI-powered review intelligence in real-world production environments.